

BHV_ZigZag, BHV_Timer, and BHV_TestFailure

Zig-zag, timer, and test-failure behaviors.

Included MIT MOOS-IvP PDF sources:

- bhv_zigzag.pdf
- bhv_timer.pdf
- bhv_testfail.pdf

The ZigZag Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/bhvdocs/bhv_zigzag

1	The ZigZag Behavior	1
1.1	Configuring the Number of Legs and Direction	2
1.2	Stem Distance and Stem Odometry	2
1.3	Configuring the Zig Leg Leg Manner	3
1.4	Configuring Speed and Speed Modifications	4
1.5	Visual Configuration Options	4
1.6	Configuration Parameters	5
1.7	Variables Published	6
1.8	Flags and Macros	6

Contents

1 The ZigZag Behavior

The *ZigZag* behavior allows a vehicle to excute a series of port-starboard turns, creating a zigzag pattern. The behavior is primarily motivated by supporting surface vehicle shake-out tests, but may be used in any mission where this pattern is desired. The behavior can be configured with varying number of turns, order of turns, and the number of degrees for each turn, and the manner in which the turn is requested.

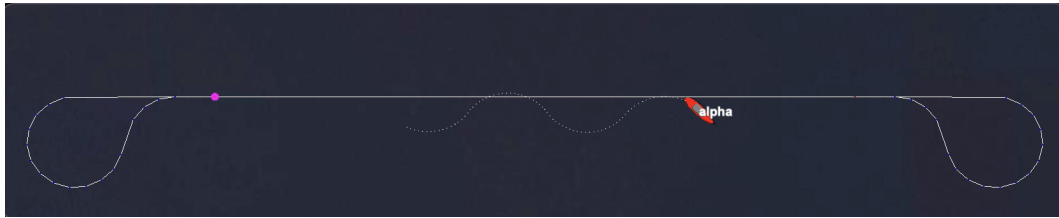


Figure 1: **ZigZag Behavior:** A vehicle executes a ZigZag behavior, with a pair of zigzags. The behavior starts and finishes within a LegRun behavior using the basic LegRun ability to initiate an event at given point in a leg. (From demo mission `s6_zigzag`)

The basic ZigZag behavior will be configured with (a) the number of zig legs desired, (b) the target change in ownship heading comprising the satisfaction of a zig leg, (c) the initial zig direction, and a few other parameters. The idea is shown in Figures 1 and 2.

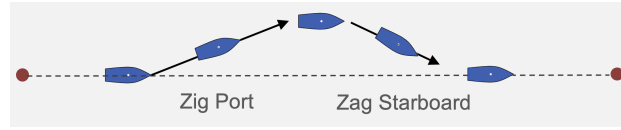


Figure 2: **ZigZag Behavior:** A zig leg is comprised of a change in heading to either the port or starboard side. Once the change of heading has been achieved, the next zig leg is executed. When all scheduled zig legs have been executed, the behavior completes.

1.1 Configuring the Number of Legs and Direction

A *zigzag* consists of two *zig legs*. The behavior can be configured by specifying the amount of zig zags or zig legs. The following two parameters are equivalent:

```
max_zig_legs = 6
max_zig_zags = 3
```

By default, the first zig leg will be a turn in the port direction. This can be overridden with:

```
zig_first = star
```

When the target number of zig legs has been completed, the behavior itself will complete, posting its end flags as any other behavior. Additionally, like any other behavior, it may be configured with `perpetual=true`, to allow additional zigzag runs. The `s6_zigzag` example mission is configured in this way.

Several macros are supported to reveal state during a mission. The total number of zig legs completed, total number of zig zags completed, current zig leg direction, total zig legs remaining. See Section 1.8.

1.2 Stem Distance and Stem Odometry

The ZigZag behavior establishes a stem line when it begins its sequence of zig legs. Two distances relative to the stem line are updated during the sequence of zig legs. The *stem distance* is the current distance of ownship perpendicular to the stem line. The *stem odometry* is the current distance of ownship parallel to the stem line as shown in Figure 3

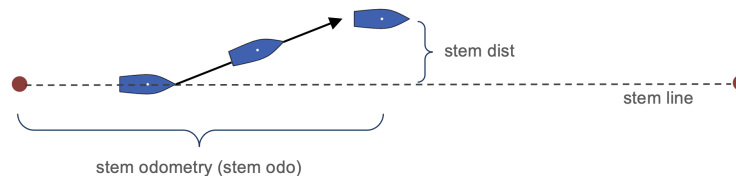


Figure 3: **Stem Line Metrics:** The stem distance and stem odometry.

The stem odometry can be optionally used as a completion criteria for the behavior (as opposed to relying solely on a maximum number of zig legs). This can be configured with:

```
max_stem_odo=100
```

With the above configuration, the behavior will complete after the max number of zig legs is reached, *or* when the stem odometry exceeds 100 meters, whichever comes first.

These two values are accessible in the macros `$(STEM_DIST)` and `$(STEM_ODO)`. As with any macro, they can be published as part of any behavior flag. For example:

```
endflag=TOTAL_STEM_ODO=$(STEM_ODO)
```

Or, for a flag available only to ZigZag behavior:

```
zigflag=CURR_STEM_DIST=$(STEM_DIST)
```

The stem odometry is stem distance are reset to zero when the behavior completes or becomes idle.

1.3 Configuring the Zig Leg Leg Manner

A zig leg is accomplished by achieving a requested change in heading. The change is relative to a *stem heading*. This stem heading value can be explicitly set to a value, e.g.,

```
stem_hdg=90
```

Alternatively, and arguably more common, the stem heading can be set to the ownship current heading when the behavior enters the running state. To use this mode:

```
stem_on_active=true
```

The delta heading is specified relative to the stem heading with the parameter:

```
zig_angle=45
```

A tolerance for achieving the heading change can be specified with the following parameter, given in degrees:

```
hdg_thresh=2
```

By default the tolerance threshold is zero degrees.

By default, the change in heading is affected by the "normal" means of continuously requesting a heading value N degrees offset from the stem heading. The rate of heading change will presumably be modulated by the vehicle, e.g., PID, controller. The heading actuator value typically approaching zero the closer the measured heading nears the requested heading.

As an alternative means, the behavior can be configured to achieve the heading change by constantly requesting a heading change of N degrees, relatively to the updated current vehicle heading as the vehicle turns. This is accomplished with the pair of parameters:

```
fierce_zigging=true  
zig_angle_fierce=10
```

In this mode, presumably the actuator setting will remain in a constant setting since the difference between the target heading and current ownship heading remains constant through the turn. Regardless of which method is used, the zig leg is still considered complete when the initial heading change has been achieved (within the tolerance of the `hdg_thresh`).

1.4 Configuring Speed and Speed Modifications

The goal speed of the ZigZag behavior is configured by setting:

```
speed = 1.5
```

The default speed is zero, essentially making this a mandatory parameter. A negative requested speed will result in a configuration error. A speed exceeding the prevailing max speed of the helm configuration will simply be clipped without an error.

The `speed` can be updated during the mission using the behavior `updates` parameter, just like any parameter for any other behavior. It may be convenient, however, to modify the speed by requesting a delta value relative to the current speed. This can be done with:

```
mod_speed = 0.2, or  
mod_speed = -0.1
```

The behavior goal speed will be modified by applying the delta value. A resulting goal speed less than zero or greater than the helm's prevailing max speed will simply be clipped without an error warning. The goal speed can be modified to the original speed setting with:

```
mod_speed = reset
```

The value stored in memory, as the "reset speed", is the value whenever the `speed` parameter is set. As discussed above, this can be set in the initial behavior configuration, or with dynamic updates.

The value of the `speed` parameter can also be configured to be set to the current ownship speed upon behavior activation:

```
spd_on_active = true
```

By default this is false. If enabled, when the behavior transitions from idle to running state, the `speed` is set to ownship speed, and this is the speed applied if `mod_speed=reset` is later applied.

1.5 Visual Configuration Options

On occasion it is useful to have a visual marker indicating the current goal heading. When enabled, the goal heading is rendered as a short line segment from the center of the vehicle as shown in Figure 4. This is enabled by setting:

```
visual_hints = draw_set_hdg = true
```

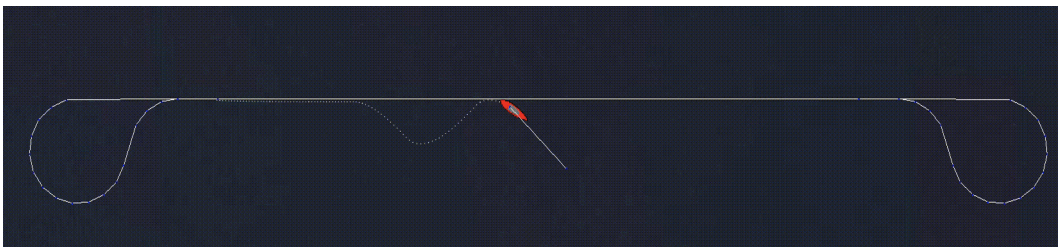


Figure 4: **ZigZag Behavior:** A vehicle executes a ZigZag behavior, with leg. (From demo mission `s6_zigzag`)

When `fierce_zigging` is enabled, the behavior uses a different heading value to achieve the goal heading. This heading can be visualized instead, or alongside, of the goal heading by setting:

```
visual_hints = draw_req_hdg = true
```

1.6 Configuration Parameters

Listing 1.1: Configuration Parameters for the ZigZag Behavior.

Parameter	Description
<code>fierce_zigging</code> :	Effect zig maneuver by requesting constant delta relative heading. Section 1.3.
<code>hdg_thresh</code> :	Angle difference to target zig heading for zig leg completion. Section 1.3.
<code>mod_speed</code> :	Apply a delta speed modification to the prevailing <code>speed</code> value. Typically applied during a mission rather than at mission startup. Section 1.4.
<code>max_zig_legs</code> :	Number of zig legs required for behavior completion. Section 1.1.
<code>max_stem_odo</code> :	Max distance parallel to the stem line, beyond which results in behavior completion, equivalent to reaching <code>max_zig_legs</code> . Section 1.1.
<code>speed</code> :	Speed in m/s to maintain during zig zag maneuvers. Default value is zero (must be set). Section 1.4.
<code>speed_on_active</code> :	The goal speed is set to the current ownship speed when the behavior becomes active. Default is <code>false</code> . Section 1.4.
<code>stem_hdg</code> :	Heading around which zig zag maneuvers will vary. Section 1.3.
<code>stem_on_active</code> :	Stem heading is set to ownship heading when the behavior becomes active. Default is <code>false</code> . Section 1.3.
<code>visual_hints</code> :	Viewing Options. See Section 1.5.
<code>zig_angle</code> :	Target relative angle required for zig leg completion. Section 1.3.
<code>zig_angle_fierce</code> :	Relative heading requested to effect a zig leg. Section 1.3.
<code>zig_first</code> :	Which side for initial zig first. Default is <code>port</code> . Other option is <code>star</code> . Section 1.1.

Listing 1.2: Example Configuration Block.

```

Behavior = BHV_ZigZag
{
    name      = zig
    pwt       = 100
    condition = MODE==ZIGZAG
    endflag   = ZIGGING = false

    updates   = ZIG_UPDATE
    perpetual = true

    speed = 2.0    // meters per sec
    stem_on_active = true
    zig_first = star
    max_zig_zags = 2
    zig_angle = 45
    zig_angle_fierce = 30

    max_stem_odo = 100

    visual_hints = draw_set_hdg = true
    visual_hints = draw_req_hdg = true

    hdg_thresh = 2
    fierce_zigging = false
}

```

1.7 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags.

- **VIEW_SEGLIST**: A visual cue for rendering the requested zig angle, if either **draw_set_hdg=true** or **draw_req_hdg=true** in the **visual_hints**. See Section 1.5.

1.8 Flags and Macros

The ZigLeg behavior supports the below set of event flags in addition to the standard behavior flags, e.g., endflags, runflags. These are:

- **zigflag**: Posted each time a zig leg is completed. These flags are posted twice as frequent as **zagflag** postings.
- **zagflag**: Posted each time a zig zag is completed. These flags will be posted half as frequent as **zigflag** postings.
- **starflag**: Posted each time a zig leg started in the starboard direction.
- **portflag**: Posted each time a zig leg started in the port direction.
- **starflagx**: Posted each time a zig leg has *completed* in the starboard direction.
- **portflagx**: Posted each time a zig leg has *completed* in the port direction.

The following macros are supported in the ZigZag behavior. These macros will be expanded in any

event flag, including event flags defined for all IvP behaviors as well as event flags defined only for the ZigZag behavior.

- `#[ODO]`: Total odometry distance since the behavior became active.
- `#[ZIG_ODO]`: Total odometry distance since the behavior began its current zig leg.
- `#[ZAG_ODO]`: Total odometry distance since the behavior began its current zig zag.
- `#[STEM_DIST]`: The current ownship distance perpendicular from the stem line. Section 1.2.
- `#[STEM_ODO]`: The current ownship distance parallel along the stem line. Section 1.2.
- `#[STEM_HDG]`: The stemline heading direction.
- `#[ZIGS]`: Total zig legs completed since behavior activation. Section 1.1.
- `#[ZAGS]`: Total zig zags completed since behavior activation. Section 1.1.
- `#[ZIGS_EVER]`: Total zig legs completed since this activation and all prior activations. For example if `perpetual=true` and repeated activations as in example mission `s6_zigzag`. Section 1.1.
- `#[ZAGS_EVER]`: Total zig zags completed since this activation and all prior activations. For example if `perpetual=true` and repeated activations as in example mission `s6_zigzag`. Section 1.1.
- `#[ZIG_START_SPD]`: Ownship speed at the start of the current zig leg.
- `#[ZIG_MIN_SPD]`: Minimum ownship speed recorded since the start of the current zig leg.
- `#[ZIG_SPD_DELTA]`: Change in ownship speed since the start of the current zig leg.
- `#[ZIGS_TOGO]`: Remaining zig legs to execute before completion, including the current zig leg being executed. Section 1.1.
- `#[ZAGS_TOGO]`: Remaining zig zags to complete before completion, including the current zig zag being executed. Section 1.1.

The Timer Behavior

Spring 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/bhvd docs/bhv_timer

1	The Timer Behavior	1
1.1	Configuration Parameters	1
1.2	Variables Published	2

Contents

1 The Timer Behavior

The Timer behavior is a somewhat unique behavior in that it never produces an objective function. It has virtually no functionality beyond what is derived from the parent IvPBehavior class. It can be used to set a timer between the observation of one or more events (with condition flags) and the posting of one or more events (with end flags). The `duration`, `duration_status`, `duration_idle_decay`, `condition`, `runflag` and `endflag` parameters are all defined generally for behaviors. There are no additional parameters defined for this behavior.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

<code>activeflag</code> :	A MOOS variable-value pair posted when the behavior is in the <i>active</i> state. [more] .
<code>condition</code> :	Specifies a condition that must be met for the behavior to be running. [more] .
<code>duration</code> :	Time in behavior will remain running before declaring completion. [more] .
<code>duration_idle_decay</code> :	When true, duration clock is running even when in the <i>idle</i> state. [more] .
<code>duration_reset</code> :	A variable-pair such as <code>MY_RESET=true</code> , that will trigger a duration reset. [more] .
<code>duration_status</code> :	The name of a MOOS variable to which the vehicle duration status is published. [more] .
<code>endflag</code> :	A MOOS variable-value pair posted when the behavior has completed. [more] .
<code>idleflag</code> :	A MOOS variable-value pair posted when the behavior is in the <i>idle</i> state. [more] .

inactiveflag:	A MOOS variable-value posted when the behavior is <i>not</i> in the <i>active</i> state. [more] .
name:	The (unique) name of the behavior. [more] .
nostarve:	Allows a behavior to assert a maximum staleness for a MOOS variable. [more] .
perpetual:	If true allows the behavior to run even after it has completed. [more] .
post_mapping:	Re-direct behavior output normally to one MOOS variable to another instead. [more] .
priority:	The priority weight of the behavior. [more] .
pwt:	Same as priority .
runflag:	A MOOS variable and a value posted when a behavior has met its conditions. [more] .
spawnflag:	A MOOS variable and a value posted when a behavior is spawned. [more] .
spawnxflag:	A MOOS variable and a value posted when a behavior is spawned. [more] .
templating:	Turns a behavior into a template for spawning behaviors dynamically. [more] .
updates:	A MOOS variable from which behavior parameter updates are read dynamically. [more] .

Listing 1.2: Example Configuration Block.

```

Behavior = BHV_Timer
{
  // General Behavior Parameters
  // -----
  name          = bhv_timer           // default
  condition     = WAIT_REQUEST=true   // example
  duration      = 120                 // example
  idleflag      = WAITING=false       // example
  runflag       = WAITING=true        // example
  runflag       = WAITED=false        // example
  endflag       = WAITED=true         // example
  updates       = TIMER_UPDATES       // example

  // Parameters specific to this behavior
  // -----
  // None
}

```

1.2 Variables Published

This behavior publishes two variables for monitoring and logging performance - **TIMER_IDLE**, **TIMER_RUNNING**.

The TestFail Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/bhvd docs/bhv_testfail

1 The TestFailure Behavior	1
1.1 Configuration Parameters	1

Contents

1 The TestFailure Behavior

The TestFailure behavior is used to test the helm in two conceivable behavior failure modes. First, it may be used to simulate a behavior that crashes and thereby results in the crash of the helm. Second, it may be used to simulate a behavior that consumes a sufficiently large enough amount of time so as cause the helm to be considered "hung" by consumers of the helm output.

Recall that the helm is compiled, with behaviors, into a single MOOS application. Although some behaviors may be compiled into shared libraries loaded at run time, thereby not requiring a recompile, all behaviors do run as part of a single helm process. *A crashed behavior results in a crashed helm.* Furthermore the helm, on each iteration, queries each participating behavior for its input. It does not do this in separate threads, and there is no timeout with a default reply should a behavior never answer. *A hung behavior results in a hung helm.* These are architecture decisions that on one hand allow a substantial amount of simplicity in the helm implementation and debugging. Furthermore, it's not clear that a graceful and safe policy exists to safely handle a rogue behavior other than to either (a) abort the mission or (b) put the vehicle in the hands of a much more conservatively configured "standby" instance of the helm, perhaps just to get the vehicle home. This behavior is used to simulate both kinds of rogue behaviors, a behavior that crashes and a behavior the hangs. The crash is implemented simply with an `assert(0)` statement, and the hang is implemented with a long for-loop.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

- activeflag:** A MOOS variable-value pair posted when the behavior is in the *active* state. [\[more\]](#).
- condition:** Specifies a condition that must be met for the behavior to be running. [\[more\]](#).
- duration:** Time in behavior will remain running before declaring completion. [\[more\]](#).

<code>duration_idle_decay</code> :	When true, duration clock is running even when in the <i>idle</i> state. [more] .
<code>duration_reset</code> :	A variable-pair such as <code>MY_RESET=true</code> , that will trigger a duration reset. [more] .
<code>duration_status</code> :	The name of a MOOS variable to which the vehicle duration status is published. [more] .
<code>endflag</code> :	A MOOS variable-value pair posted when the behavior has completed. [more] .
<code>idleflag</code> :	A MOOS variable-value pair posted when the behavior is in the <i>idle</i> state. [more] .
<code>inactiveflag</code> :	A MOOS variable-value posted when the behavior is <i>not</i> in the <i>active</i> state. [more] .
<code>name</code> :	The (unique) name of the behavior. [more] .
<code>noSTARVE</code> :	Allows a behavior to assert a maximum staleness for a MOOS variable. [more] .
<code>perpetual</code> :	If true allows the behavior to to run even after it has completed. [more] .
<code>post_mapping</code> :	Re-direct behavior output normally to one MOOS variable to another instead. [more] .
<code>priority</code> :	The priority weight of the behavior. [more] .
<code>pwt</code> :	Same as <code>priority</code> .
<code>runflag</code> :	A MOOS variable and a value posted when a behavior has met its conditions. [more] .
<code>spawnflag</code> :	A MOOS variable and a value posted when a behavior is spawned. [more] .
<code>spawnxflag</code> :	A MOOS variable and a value posted when a behavior is spawned. [more] .
<code>templating</code> :	Turns a behavior into a template for spawning behaviors dynamically. [more] .
<code>updates</code> :	A MOOS variable from which behavior parameter updates are read dynamically. [more] .

Listing 1.2: Configuration Parameters for the TestFail Behavior.

Parameter	Description
<code>failure_type</code> :	Fail by <code>crash</code> , or fail by <code>hang</code> .

Choosing to fail with a *crash* will result in behavior executing an `assert(0)` as soon as the behavior enters the running state. Configuring for a *hang* will likewise result in the execution of a long for-loop upon entering the running state. The hang time is by default three seconds, but may be altered by specifying a time alongside the hang parameter as in line 9 below.

Below are a couple typical usage configurations. In the first case, the TestFailure behavior is set with a condition as on line 4, to remain idle until the vehicle is deployed. The duration parameter on line 6, defined for all helm behaviors, ensures the behavior will not run until two minutes into the mission. This is done presumably to have the failure occur while the vehicle is "in the middle of doing something". Setting the `duration_idle_decay` parameter to false indicates the duration countdown is not to proceed while the behavior is still idle. In this case the failure type is set to

hang for two seconds. This behavior configuration is used in the Kilo example mission, described in Section XYZ, and can be run to illustrate.

Listing 1.3: Example Configuration Block.

```
Behavior = BHV_TestFailure
{
  // General Behavior Parameters
  // -----
  name      = test_failure      // default
  condition = DEPLOY=true       // example
  duration  = 120                // example
  duration_idle_decay = false    // example (default is true)

  // Parameters specific to this behavior
  // -----
  failure_type = hang,2         // example
}
```

Another configuration style below will result in the behavior crashing as soon as the behavior meets its run condition. Presumably the user can simply poke the MOOSDB at any time to invoke the failure, or another MOOS app may generate the poke at the key desired moment. The `failure_type` is left unspecified since `crash` is the default failure type.

Listing 1.4: Example Configuration Block.

```
Behavior = BHV_TestFailure
{
  // General Behavior Parameters
  // -----
  condition = MUD_HITS_THE_FAN = true
}
```